PROGRAMACION CONCURRENTE

Monitores

la noción de un monitor fue presentada inicialmente por
dijkstra (1971),

posteriormente por
brinch hansen (1973)

y después fue refinada por
hoare (1974).

Introducción

- Los semáforos son primitivas con las cuales es difícil expresar una solución a grandes problemas de concurrencia y su presencia en programas concurrentes incrementa la ya existente dificultad para probar que los programas son correctos.
- Los semáforos tienen algunas debilidades:
  - La omisión de una de estas primitivas puede corromper la operación de un sistema concurrente.
  - El control de la concurrencia es responsabilidad del programador.
  - Las primitivas de control se encuentran esparcidas por todo el sistema.

Mas Inconveniente de los semáforos

- Nos puede llevar fácilmente a errores
  - Errores transitorios
  - La ejecución de cada sección critica sobre un semáforo debe:
    - comenzar con wait() y
    - terminar con signal() No se puede restringir el tipo de operación sobre el recurso
  - Se debe incluir todas las sentencias criticas en la sección critica
  - Tanto la sección critica como la sincronización se implementan usando las mismas primitivas y luego son difíciles de identificar
  - Los programas que usan semáforos son difíciles de mantener,
    - El código de sincronización esta repartido entre diferentes procesos

Que buscamos?

- Compartir Recursos
  - La especificación de recursos compartidos nos permite:
    - Definir la interacción entre procesos de independientemente del lenguaje o técnica de programación
    - La comunicación entre procesos se realiza mediante las operaciones sobre el recurso compartido
    - La exclusión mutua se produce entre todas las operaciones del recurso
    - La sincronización por condición se define a través de las CPRE's de las operaciones del recurso compartido
  - Pregunta
    - ¿como implementamos un recurso compartido especificado?
      - Métodos synchronized
      - Monitores
      - Paso de Mensajes

Definición

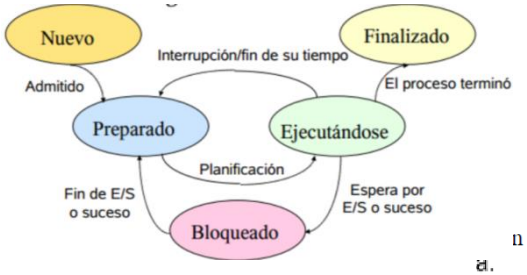
- Un monitor :
  - Es un mecanismo de software (de alto nivel) para control de concurrencia que contiene los datos y los procedimientos necesarios para realizar la asignación de un determinado recurso o grupo de recursos compartidos *reutilizables en serie.*
  - *Un monitor contiene*
    - *Variables que representan el estado del recurso*
    - *Procedimientos que implementan operaciones sobre el recurso*
  - Se usa para manejar todas las funciones de concurrencia:
    - comunicación entre procesos
    - localización física de recursos en una región crítica
  - Podríamos decir que un monitor es una instancia de una clase que puede ser usada de forma segura por múltiples threads

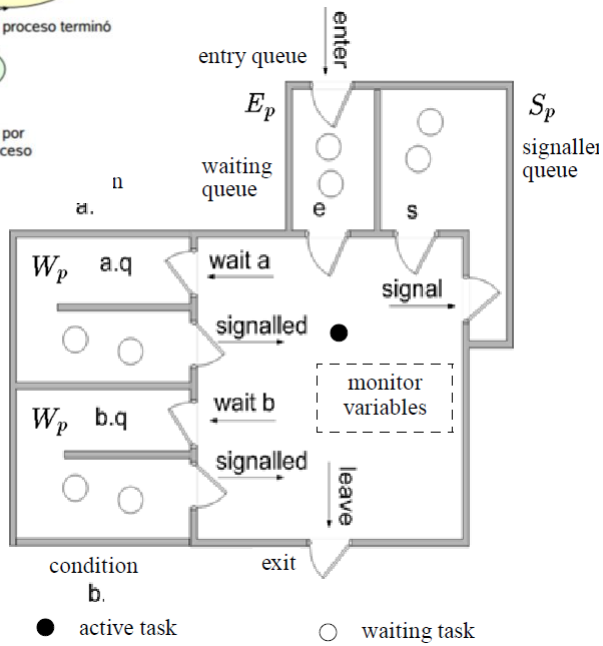
Como Hacerlo

- El monitor tiene varios procedimientos que manipulan datos internos y existe una parte de inicialización .
  - El monitor puede ser visto como una aduana en donde se permite o no el acceso a un recurso compartido o se espera por un tramite.
- El código del monitor consta de 2 partes lógicas:
  - El algoritmo para la manipulación del recurso y sincronización.
  - El mecanismo para la asignación del orden en el cual los procesos asociados pueden compartir el recurso y/o son sincronizados.
- Desde el punto de vista de los procesos activos que usan al monitor, este ofrece un conjunto de procedimientos para utilizar recursos compartidos y sincronizar actividades
- Se cumple que:
  - Todos los métodos de un monitor deben ejecutarse en exclusión mutua
  - Se define la sincronización por condición de cada uno de los métodos del monitor
    - bloquear procesos hasta que se cumplan las condiciones para que puedan ejecutar
    - notificar (señalizar) a un proceso bloqueado que puede continuar su ejecución cuando se cumplan las condiciones para ejecutarse

Estructura básica de un monitor

- Para asegurar que un proceso obtenga el recurso que espera, el monitor *(la lógica del monitor) debe* darle prioridad sobre los nuevos procesos que solicitan entrar al monitor.
- De otra manera, los nuevos procesos tomaran el recurso antes de que el proceso que espera lo haga, esto puede llevar al proceso que espera a la *postergación indefinida.*





Elementos que componen al Monitor

- Conjunto de **variables locales**, denominadas permanentes, almacenan el estado interno del recurso, y los procedimientos
- Código de inicialización (**constructo**r)
- Conjunto de **procedimientos interno** que maneja las variables permanentes
- Declaración de los **procedimientos** que son **exportados** pueden ser accedidos por los procesos activos

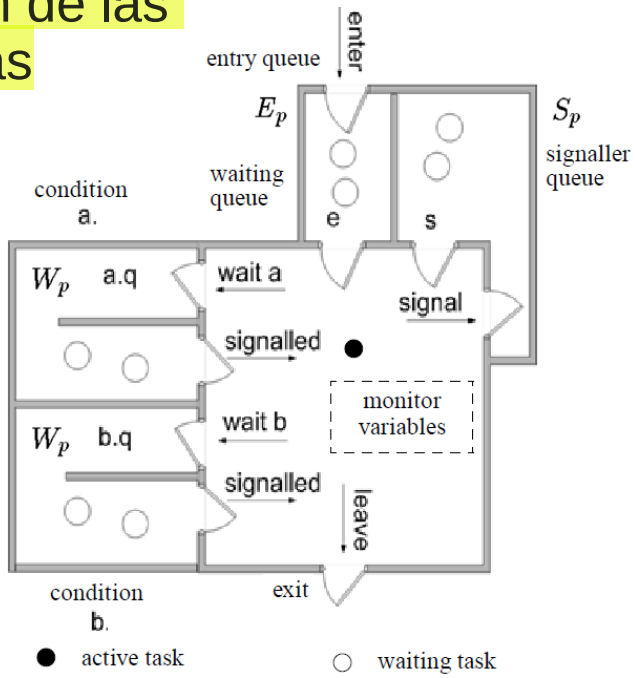
```
Monitor nombre_monitor
        var variables_locales;
        export procedimientos_exportados;

        procedure proc1(parametros){
                var variables locales;
                begin
        // Codigo del procedimiento
                end

        procedure proc2(parametros){
                var variables locales;
                begin
        // Codigo del procedimiento
                end
begin
        // codigo de inicializacion
end
```

Gestión del Acceso Exclusivo

- El control de la exclusión mutua esta basado en la **cola asociada del monitor**
- Gestión de la Cola
  - Un proceso activo esta ejecutando un procedimiento del monitor (el procedimiento esta en el monitor) y otro proceso activo trata de ejecutar el miso u otro, el código de acceso al monitor bloquea la llamada y lo manda a la cola
  - Cuando un proceso activo abandona el monitor , el monitor toma el proceso que esta el frente de la cola y lo desbloquea
  - Si la cola del monitor esta vacía, este queda libre
- Lo anterior garantiza la exclusión mutua, el responsable de esto es el monitor

Clasificación de Monitores

- Los monitores se dividen en función de señal la operación:
  - señal explícita
    - Es un monitor con una declaración explícita de la variable señal
  - Señal implícita
    - Es un monitor de señal automática, que no tiene ninguna declaración
  - A su vez, la clasificación esta basada en función de las prioridades relativas asociadas a estas tres colas
    - Prioridad relativa de la cola de entrada
    - Prioridad relativa de la cola de espera
    - Prioridad relativa de la cola de señalización.
  - Las políticas que nos interesan son:
    - Wait and Notify $E_p = W_p < S_p$
    - Signal and Wait $E_p = S_p < W_p$
    - Signal and Continue $E_p < W_p < S_p$
    - Signal and Urgent $E_p < S_p < W_p$



Monitores: Polticas Signal

- **Signal and Continue (SC):** El proceso que señaliza mantiene el mutex y el proceso despertado debe competir por el mutex para ejecutar

$$E_p < W_p < S_p$$

- **Signal and Wait (SW):** El proceso que señaliza es bloqueado y debe competir de nuevo por el mutex para continuar y el proceso despertado adquiere el mutex y continua su ejecución

$$E_p = S_p < W_p$$

- **Signal and Urgent Wait (SU):** El proceso que señaliza es bloqueado pero será el primero en conseguir el mutex cuando lo libere el proceso despertado

$$E_p < S_p < W_p$$

- **Signal and Exit (SX):** El proceso que señaliza sale del metodo y el proceso despertado coge directamente el mutex para ejecutar

$$E_p = W_p < S_p$$

Monitor: Variable de Condición

- Una **variable de condición** son “contenedores” de subprocesos que esperan una determinada condición.
  - Los monitores proporcionan un mecanismo para que los subprocesos otorguen temporalmente acceso exclusivo para esperar a que se cumpla alguna condición, antes de recuperar el acceso exclusivo y reanudar su tarea.
    - Cada proceso puede requerir una sincronización distinta, por lo que hay que programar cada caso, para ello, se usarán las variables “condición”
      - se usan para hacer esperar a un proceso hasta que determinada condición sobre el estado del monitor se “cumpla”
      - también para despertar a un proceso que estaba esperando por su causa
  - instrucción **wait:**
    - el proceso invocador de la acción que la contiene queda “dormido”
    - y pasa a la cola asociada a la variable, en espera de ser despertado
  - instrucción **signal**:
    - si la **cola** de la señal está **vacía**: no pasa nada, y la acción que la ejecuta sigue con su ejecución
      - al terminar, el monitor está disponible para otro proceso
    - si la **cola no** está **vacía**:
      - el primer proceso de la cola se despierta (pero no avanza) se saca de la cola
      - el proceso que la ejecuta sigue con su ejecución, hasta terminar el procedimiento

Depende de la política implementada- Schedule

Polticas Signal: Signal and Continue (SC)

- Con las *variables de condición sin bloqueo* , *"señal y continuar"*
  - la señalización no hace que el hilo de señalización pierda la ocupación del monitor.
  - En cambio, los hilos señalizados se mueven a la cola de entrada. No hay necesidad de cola de cortesía.

$$E_p < W_p < S_p$$

```
notify all c :
   move all threads waiting on c.q to e
```

mueve todos los subprocesos que esperan una variable de condición a la cola de entrada y la da mas prioridad que los que esperan para entrar.

```
enter the monitor:
   enter the method
   if the monitor is locked
     add this thread to e
     block this thread
   else
     lock the monitor
```

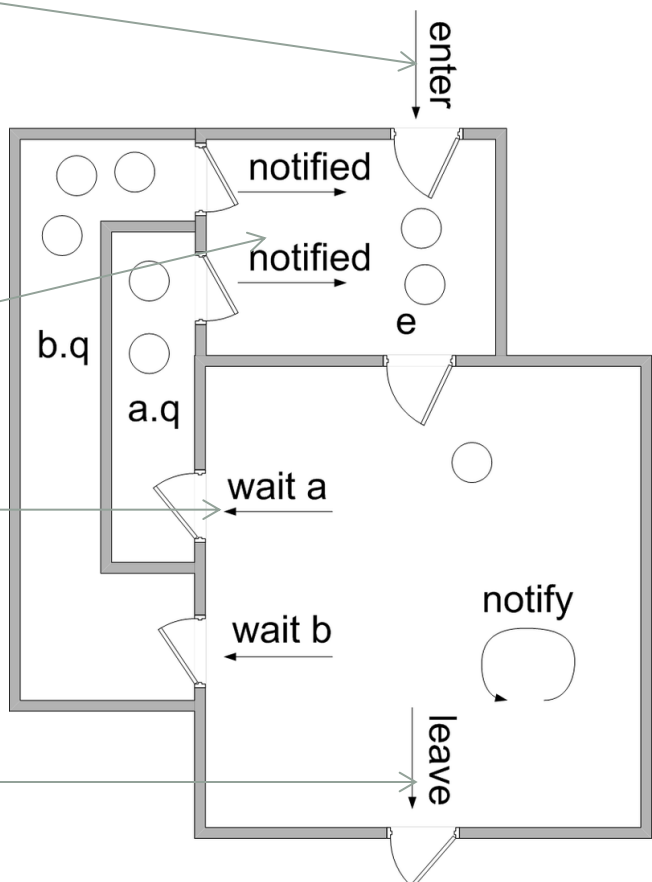
La cola de entrada le tiene que asignar mas prioridad a los procesos que son movidos desde t.

```
notify c :
   if there is a thread waiting on c.q
       select and remove one thread t from c.q
       (t is called "the notified thread")
       move t to e
```

```
schedule :
  if there is a thread on e
    select and remove one thread from e and restart it
  else
    unlock the monitor
```

```
wait c :
   add this thread to c.q
   schedule
   block this thread
```

```
leave the monitor:
   schedule
   return from the method
```



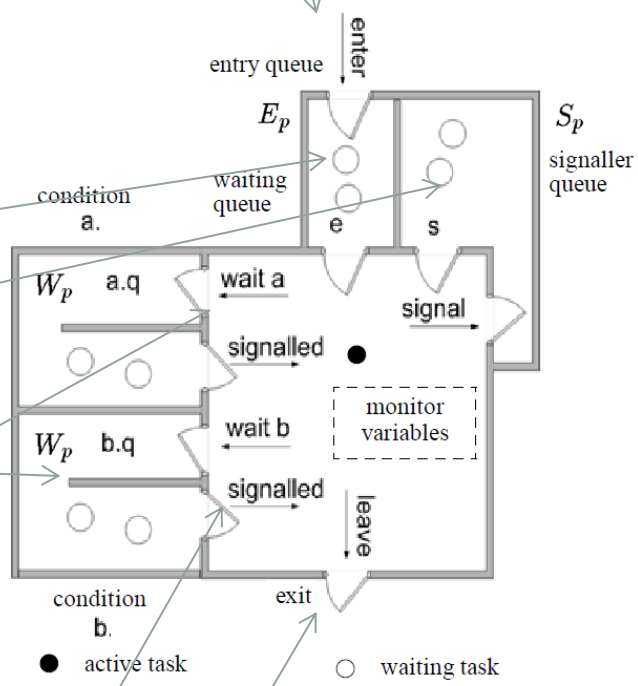
Polticas Signal:

- Signal and Urgent Wait (SU) $\bar{E_p} < \bar{S_p} < \bar{W_p}$

La disciplina de señalización resultante se conoce como *"señal y espera urgente"*, ya que el señalizador debe esperar, pero se le da prioridad sobre los hilos en la cola de entrada.
Una alternativa es *"señal y espera"*, en la que no hay s-cola y el señalador espera en la e-cola.

- hay dos colas de subprocesos asociados con cada objeto en el monitor
  - e es la cola de entrada
  - s es una cola de hilos que han señalado.
- para cada variable de condición c , hay una cola
  - c.q, que es una cola para subprocesos que esperan en la variable de condición c

```
enter the monitor:
   enter the method
   if the monitor is locked
       add this thread to e
       block this thread
   else
       lock the monitor
```

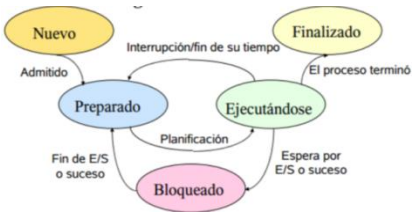



```
schedule :
  if there is a thread on s
    select and remove one thread from s and restart it
    (this thread will occupy the monitor next)
  else if there is a thread on e
    select and remove one thread from e and restart it
    (this thread will occupy the monitor next)
  else
    unlock the monitor
    (the monitor will become unoccupied)
```

```
wait c :
   add this thread to c.q
   schedule
   block this thread
```

```
leave the monitor:
   schedule
   return from the method
```

```
signal c :
   if there is a thread waiting on c.q
       select and remove one such thread t from c.q
       (t is called "the signaled thread")
       add this thread to s
       restart t
       (so t will occupy the monitor next)
       block this thread
```



Como Wp se transfiere la urgencia a Wp

Proceso	Estado	Proceso	Estado
s	sig eje	a	blo
s	wai eje	a	pre
s	blo	sig a	eje
s	pre	a	eje

Polticas Signal:

- Signal and Wait (SW)

$$E_p = S_p < W_p$$

mueve los subprocesos que esperan una variable de condición al monitor como activo.
El proceso señalizador a la cola de espera, donde la prioridad es la de llegada.

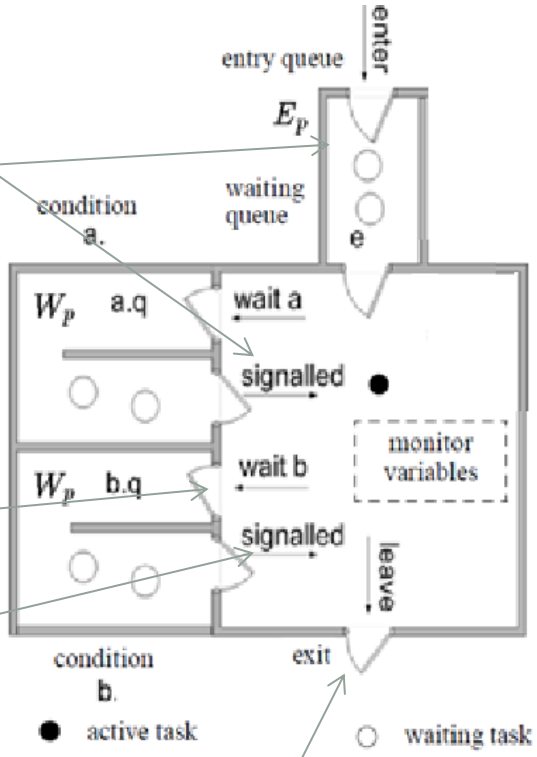
- Signal and Exit (SX):

$$E_p = W_p < S_p$$

mueve los subprocesos que esperan una variable de condición a la cola de entrada, con la prioridad de llegada.
El proceso señalizador sigue en el monitor hasta terminar, al salir llama a schedule.

```
enter the monitor:
   enter the method
   if the monitor is locked
       add this thread to e
       block this thread
   else
       lock the monitor
```

```
signal c
   if there is a thread waiting on c.q
       select and remove one thread t from c.q
       (t is called "the notified thread")
       move t to e
   else
       schedule
   return from the method
```




```
wait c :
   add this thread to c.q
   schedule
   block this thread
```

```
signal c and return :
   if there is a thread waiting on c.q
       select and remove one such thread t from c.q
       (t is called "the signaled thread")
       restart t
       (so t will occupy the monitor next)
   else
       schedule
   return from the method
```

```
schedule :
  if there is a thread on e
    select and remove one thread from e and restart it
  else
    unlock the monitor
```

```
leave the monitor:
   schedule
   return from the method
```

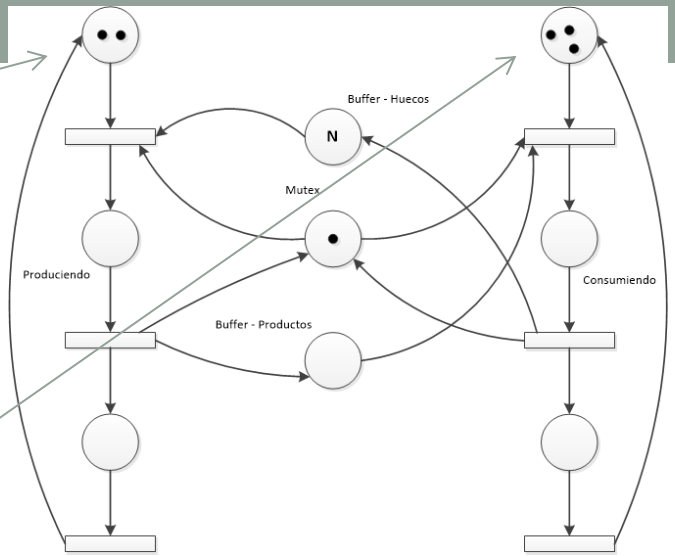
Objetivo

- Nuestro objetivo solucionar un problemas de concurrencia que resultarían muy difíciles con semáforos.
- Hemos visto que las colas de monitores y condiciones interactúan de maneras no triviales.
- La solución que proponemos es un monitores que facilite identificar, separar y gestionar:
  - La lógica del Sistema
  - La política del Sistema
    - Que emerge de la relación entre las colas de entrada, condición y cortesía
    - De las colas de condición
  - Las acciones que se ejecutan
- Concretamente, tendremos que decidir:
  1. una estructura general para los métodos que implementan acciones de un recurso compartido (en el monitor),
  2. que condición en las colas se requiere para los bloqueos y
  3. una estrategia para efectuar los desbloqueos cuando hay varios hilos candidatos a ser desbloqueados.

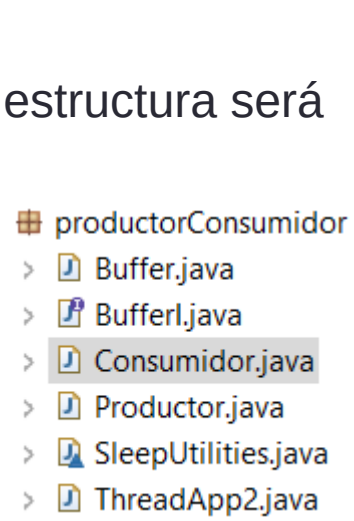
Métodos en el monitor

```java
4  public class Productor implements Runnable {
5      /**
6       * This is the producer thread for the bounded buffer problem.
7       */
8
9
10          private  Buffer buffer;
11
12          public Productor(Buffer b) {
13            buffer = b;
14          }
15
16          public void run(){
17            Date message;
18            while (true) {
19               System.out.println("Producer napping");
20               SleepUtilities.nap();
21            // produce an item & enter it into the buffer
22               message = new Date();
23               System.out.println("Producer produced \"" + message + "\"");
24               buffer.insert(message);
25            }
26          }
27      }
```

```java
8
9           private Buffer buffer;
10
11          public Consumidor(Buffer b) {
12            buffer = b;
13          }
14
15          public void run(){
16            Date message = null;
17            while (true){
18               System.out.println("Consumer napping");
19               SleepUtilities.nap();
20            // consume an item from the buffer
21               System.out.println("Consumer wants to consume");
22               message = (Date)buffer.remove();
23               System.out.println("Consumer consumed \"" + message + "\"");
24            }
25          }
26      }
```

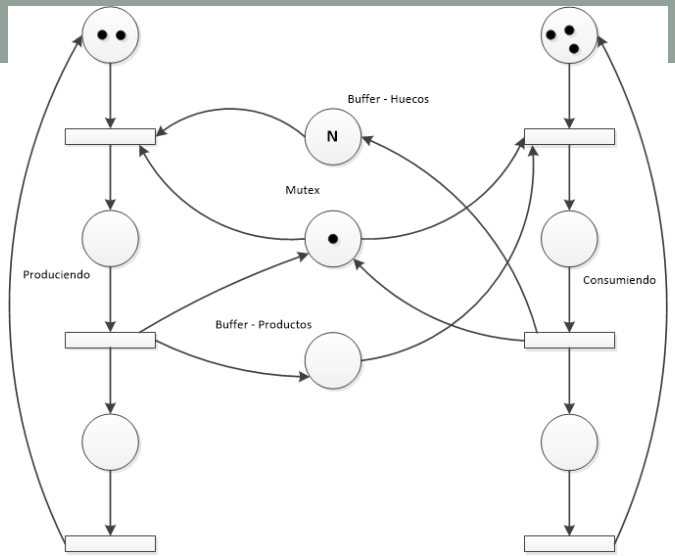


- La estructura será



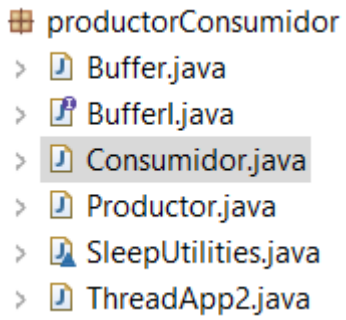
Métodos en el monitor

```java
4  public class ThreadApp2 {
5
6      public static void main(String args[]) {
7          //instantiate (create) buffer shared by Producer & Consumer
8          Buffer sharedBuffer = new Buffer();
9          // create the producer and consumer threads
10         Thread producerThread = new Thread(new Productor(sharedBuffer));
11         Thread consumerThread = new Thread(new Consumidor(sharedBuffer));
12         //start() method allocates memory for a new thread in the JVM,
13         //and calls the run() method
14         producerThread.start();
15         consumerThread.start();
16     }
17 }
18
```

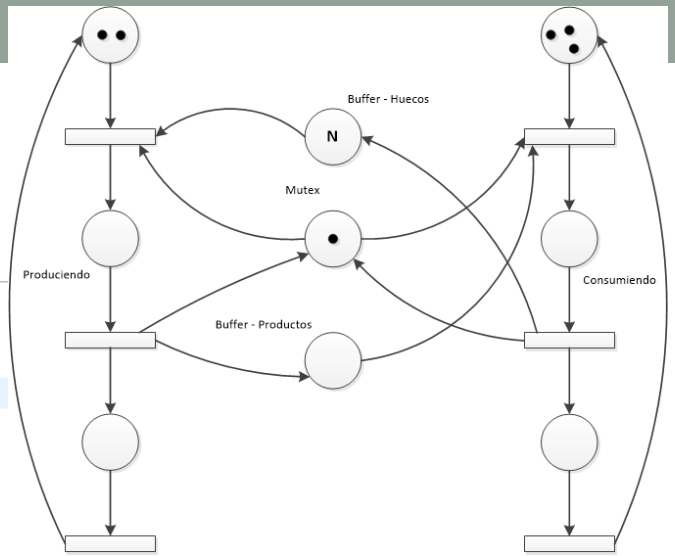



```java
10 class SleepUtilities{
11
12     private static final int NAP_TIME = 3; //max nap time in seconds
13
14     /**
15      * Nap between zero and NAP_TIME seconds.
16      */
17     public static void nap() {
18         nap(NAP_TIME);
19     }
20
21     /**
22      * Nap between zero and duration seconds.
23      */
24     public static void nap(int duration) {
25         int sleeptime = (int) (NAP_TIME * Math.random() );
26         System.out.println("Nap for " + sleeptime + " seconds");
27         //Causes the currently executing thread to sleep (cease execution)
28         //for the specified number of milliseconds,
29         //subject to the precision and accuracy of system timers and schedulers.
30         try { Thread.sleep(sleeptime*1000); }
31         catch (InterruptedException e) {
32             //method sleep() throws InterruptedException - if any thread has interrupted the current thread.
33             System.out.println("ERROR in nap(): " + e);
34         }
35     }
36 }
37
```

- La estructura será

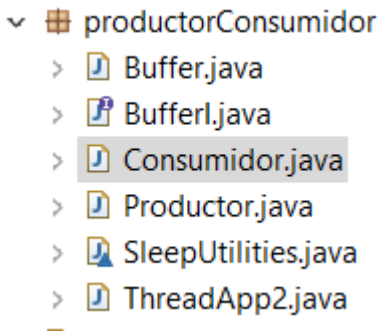


Métodos en el monitor



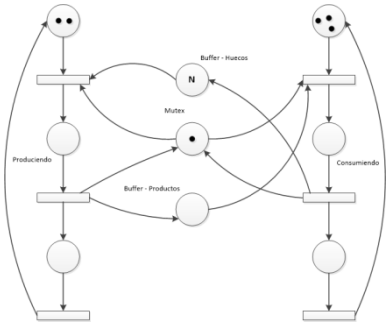

```java
package productorConsumidor;
import java.util.concurrent.*;

public class Buffer implements BufferI{
    private static final int BUFFER_SIZE = 3; //max size of buffer array
    private int count; //number of items currently in the buffer
    private int in;   // points to the next free position in the buffer
    private int out;  // points to the first filled position in the buffer
    private Object[] buffer; //array of Objects
    private Semaphore mutex; //provides limited access to the buffer (mutual exclusion)
    private Semaphore empty; //keep track of the number of empty elements in the array
    private Semaphore full; //keep track of the number of filled elements in the array
    public Buffer(){
        // buffer is initially empty
           count = 0;
           in = 0;
           out = 0;
           buffer = new Object[BUFFER_SIZE];
           mutex = new Semaphore(1); //1 for mutual exclusion
           empty = new Semaphore(BUFFER_SIZE); //array begins with all empty elements
           full = new Semaphore(0); //array begins with no elements
        }
```

- La estructura será



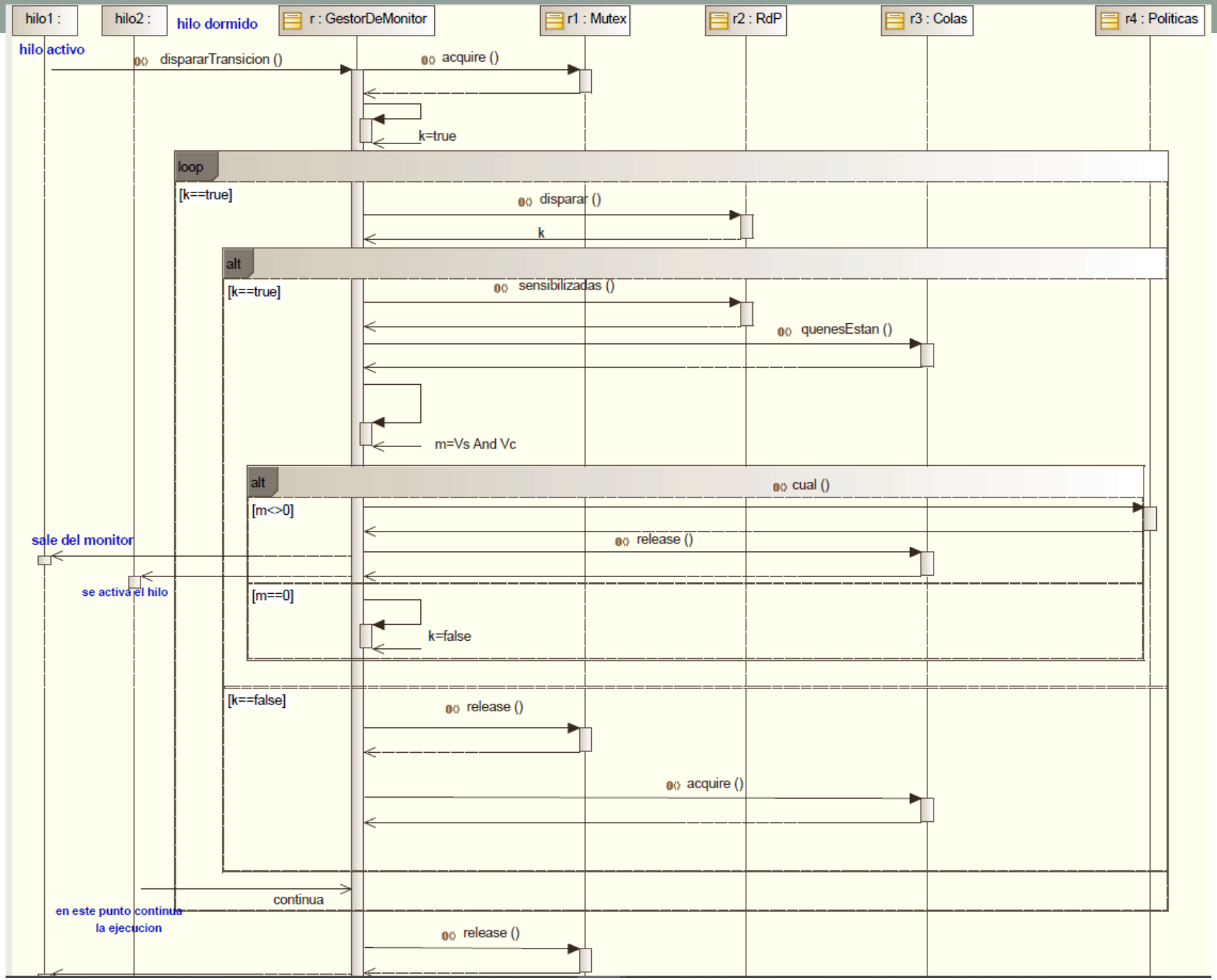
```java
    public void insert(Object item) {
    /*while (count == BUFFER_SIZE){
        // do nothing, if the buffer array cannot be used (because full)
      } */
      try{
         empty.acquire(); //keep track of number of empty elements (value--)
         //This provides synchronization for the producer,
       //because this makes the producer stop running when buffer is full
         mutex.acquire(); //mutual exclusion
      }
          catch (InterruptedException e) {
            System.out.println("ERROR in insert(): " + e);
         }

   // add an item to the buffer
      ++count;
      buffer[in] = item;
   //modulus (%) is the remainder of a division
   //for example, 0%3=0, 1%3=1, 2%3=2, 3%3=0, 4%3=1, 5%3=2
      in = (in + 1) % BUFFER_SIZE;

   //buffer information feedback
      if (count == BUFFER_SIZE){
         System.out.println("BUFFER FULL "
            + "Producer inserted \"" + item
            + "\" count=" + count + ", "
            + "in=" + in + ", out=" + out);
      }
      else{
         System.out.println("Producer inserted \"" + item
            + "\" count=" + count + ", "
            + "in=" + in + ", out=" + out);
      }

      mutex.release(); //mutual exclusion
      full.release(); //keep track of number of elements (value++)
      //If buffer was empty, then this wakes up the Consumer
   }
```

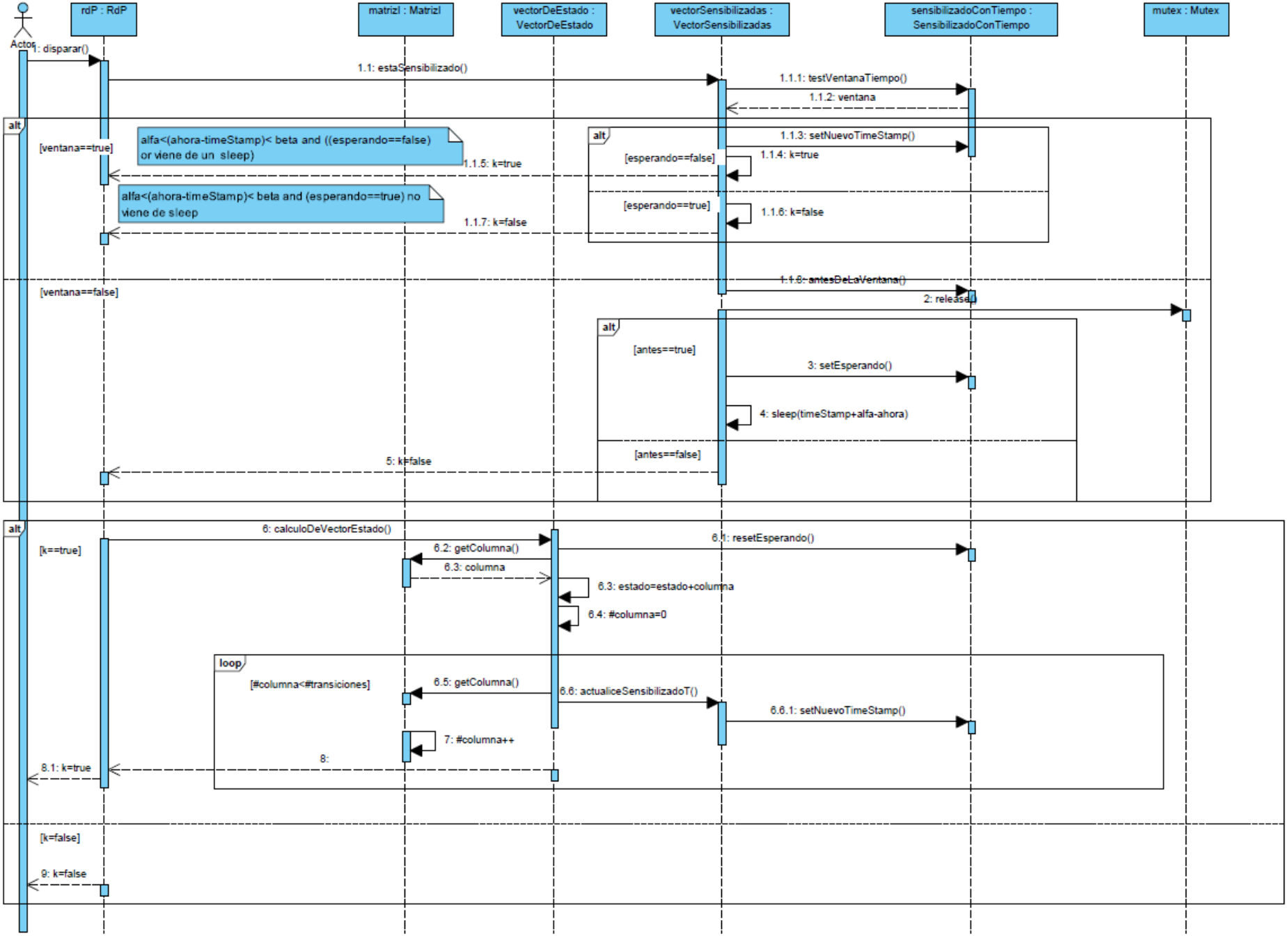
```java
// consumer calls this method
    public Object remove() {
      Object item=null;
   /* while (count == 0){
      //if nothing in the buffer, then do nothing
      //the buffer array cannot be used (because empty)
      } */

      try{
         full.acquire(); //keep track of number of elements (value--)
         //This provides synchronization for consumer,
       //because this makes the Consumer stop running when buffer is empty
         mutex.acquire(); //mutual exclusion
      }
          catch (InterruptedException e) {
            System.out.println("ERROR in try(): " + e);
         }
   // remove an item from the buffer
      --count;
      item = buffer[out];
   //modulus (%) is the remainder of a division
   //for example, 0%3=0, 1%3=1, 2%3=2, 3%3=0, 4%3=1, 5%3=2
      out = (out + 1) % BUFFER_SIZE;
   //buffer information feedback
      if (count == 0){
         System.out.println("BUFFER EMPTY "
            + "Consumer removed \"" + item
            + "\" count=" + count + ", "
            + "in=" + in + ", out=" + out);
      }
      else{
         System.out.println("Consumer removed \"" + item
            + "\" count=" + count + ", "
            + "in=" + in + ", out=" + out);
      }
      mutex.release(); //mutual exclusion
      empty.release(); //keep track of number of empty elements (value++)
       //if buffer was full, then this wakes up the Producer
      return item;
   }

}
```




```java
public interface BufferI {
    /**
        * insert an item into the Buffer.
        * Note this may be either a blocking
        * or non-blocking operation.
        */
           public abstract void insert(Object item);

       /**
        * remove an item from the Buffer.
        * Note this may be either a blocking
        * or non-blocking operation.
        */
           public abstract Object remove();
}
```

- productorConsumidor
  - Buffer.java
  - BufferI.java
  - Consumidor.java
  - Productor.java
  - SleepUtilities.java
  - ThreadApp2.java


hilo1 :
hilo2 :
hilo dormido
r : GestorDeMonitor
r1 : Mutex
r2 : RdP
r3 : Colas
r4 : Politicas
hilo activo
dispararTransicion ()
acquire ()
k=true
loop
[k==true]
disparar ()
k
alt
[k==true]
sensibilizadas ()
quenesEstan ()
m=Vs And Vc
alt
cual ()
[m<>0]
release ()
sale del monitor
se activa el hilo
[m==0]
k=false
[k==false]
release ()
acquire ()
continua
en este punto continua
la ejecucion
release ()


Actor
rdP : RdP
matrizI : MatrizI
vectorDeEstado : VectorDeEstado
vectorSensibilizadas : VectorSensibilizadas
sensibilizadoConTiempo : SensibilizadoConTiempo
mutex : Mutex
1: disparar()
1.1: estaSensibilizado()
1.1.1: testVentanaTiempo()
1.1.2: ventana
alt
[ventana==true]
alfa<(ahora-timeStamp)< beta and ((esperando==false) or viene de un sleep)
1.1.5: k=true
alfa<(ahora-timeStamp)< beta and (esperando==true) no viene de sleep
1.1.7: k=false
alt
1.1.3: setNuevoTimeStamp()
[esperando==false]
1.1.4: k=true
[esperando==true]
1.1.6: k=false
[ventana==false]
1.1.8: antesDeLaVentana()
2: release()
alt
[antes==true]
3: setEsperando()
4: sleep(timeStamp+alfa-ahora)
[antes==false]
5: k=false
alt
[k==true]
6: calculoDeVectorEstado()
6.1: resetEsperando()
6.2: getColumna()
6.3: columna
6.3: estado=estado+columna
6.4: #columna=0
loop
[#columna<#transiciones]
6.5: getColumna()
6.6: actualiceSensibilizadoT()
6.6.1: setNuevoTimeStamp()
7: #columna++
8:
8.1: k=true
[k=false]
9: k=false

Bibliografía

Notas de clase de Programación Concurrente

Comunicación y Sincronización con Monitores

Resumen del Tema

Dpto de Lenguajes y Ciencias de la Computación Universidad de Málaga

María del Mar Gallardo Melgarejo

Málaga, 2002